Experiment 1: To build a Gaussian Naive Bayes classifier using the Iris dataset and evaluate its accuracy.

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, confusion_matrix
iris = load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.3)
model = GaussianNB()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
```

Experiment 2: To train a Decision Tree classifier on the Iris dataset and visualize the learned tree.

```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier, plot_tree
import matplotlib.pyplot as plt

iris = load_iris()
X, y = iris.data, iris.target
model = DecisionTreeClassifier(max_depth=3)
model.fit(X, y)
print("Prediction:", model.predict([X[0]]))
print("Accuracy:", model.score(X, y))
plot_tree(model, filled=True)
plt.show()
```

Experiment 3: To implement Linear Regression for predicting a numerical value and Logistic Regression for binary classification.

```
#Part A : Linear Regression
from sklearn.linear_model import LinearRegression
X = [[1], [2], [3], [4], [5]]
y = [20, 30, 40, 50, 60]
model = LinearRegression()
model.fit(X, y)
print("Predicted marks:", model.predict([[6]])[0])
```

```
# Part B: Logistic Regression - predict Pass (1) or Fail (0)
from sklearn.linear_model import LogisticRegression

X = [[20], [30], [40], [60], [70], [80]]
y = [0, 0, 0, 1, 1, 1]
model = LogisticRegression()
model.fit(X, y)
print("Prediction:", model.predict([[65]])[0])
print("Accuracy:", model.score(X, y))
```

Experiment 4 : To implement a Random Forest classifier and evaluate its performance using accuracy and a confusion matrix.

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix
 from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix

iris = load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.3)
model = RandomForestClassifier(n_estimators=10)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
```

Experiment 5 : To group data points into two clusters using K-Means and plot the resulting clusters.

```
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans

X = [[1,2], [2,3], [3,2], [8,8], [9,8], [8,9]]

k = KMeans(n_clusters=2, n_init=10)
k.fit(X)
print("Labels:", k.labels_)
plt.scatter([p[0] for p in X], [p[1] for p in X], c=k.labels_)
plt.show()
```

Experiment 6A: To perform agglomerative hierarchical clustering using complete linkage and display a dendrogram.

```
import matplotlib.pyplot as plt
from scipy.cluster.hierarchy import dendrogram, linkage

X = [[1, 2], [2, 3], [3, 2], [8, 8], [9, 8], [8, 9]]

Z = linkage(X, method="complete")
dendrogram(Z)

plt.title("Hierarchical Clustering Dendrogram")
plt.xlabel("Data Point")
plt.ylabel("Distance")
plt.show()
```

Experiment 6B:To identify clusters and noise points using the DBSCAN clustering algorithm.

```
import matplotlib.pyplot as plt
from sklearn.cluster import DBSCAN

X = [[1,1], [1,2], [2,1], [2,2], [7,7], [7,8], [8,7], [10,2]]
model = DBSCAN(eps=1.5, min_samples=3)
labels = model.fit_predict(X)

print("Labels:", labels)
plt.scatter([p[0] for p in X], [p[1] for p in X], c=labels)
plt.show()
```

Experiment 7: To reduce the Iris dataset from four features to two principal components and visualize the reduced data.

```
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

iris = load_iris()
X = StandardScaler().fit_transform(iris.data)
pca = PCA(n_components=2)
X_new = pca.fit_transform(X)
print("Reduced shape:", X_new.shape)
plt.scatter(X_new[:, 0], X_new[:, 1], c=iris.target)
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.show()
```

Experiment 8: To train a single-layer Perceptron for binary classification and evaluate it using accuracy and a confusion matrix.

```
from sklearn.datasets import load_iris
from sklearn.linear_model import Perceptron

iris = load_iris()
X = iris.data[:100, :2]
y = iris.target[:100]
model = Perceptron()
model.fit(X, y)
print("Prediction:", model.predict([[5, 3]]))
```